

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO-COMPUTADORA

SEMESTRE 2027 - 1



TEXTURIZADO, COLOR E ILUMINACIÓN

ALUMNOS	*Basilio Illescas Andrés*	*Product Owner*
	Trejo Olvera Emmanuel	*Scrum Master*
	Pérez Olvera Alexis Abraham	*Developer*
	Cruz Macedo Samuel Santiago	*Developer*
PROFESOR	*Ing. José Ramón Pérez Athié*	
GRUPO	*04*	

Ciudad Universitaria, CDMX

28 de octubre de 2026

Índice

1. Introducción **2**

2. Objetivos **2**

2.1. Objetivo general 2

2.2. Objetivos específicos 2

3. Alcance de la entrega **2**

4. Metodología de trabajo ágil **3**

5. Diseño y desarrollo **3**

5.1. Flujo de Trabajo en Blender: Texturas Procedurales 3

5.2. Mapeo de Normales y Materiales PBR 4

5.3. Generación Procedural de UVs y Recorte Alfa 4

5.4. Iluminación Híbrida y Transmisión 4

5.5. Entorno Atmosférico (Skybox) 4

6. Retos arquitectónicos y compatibilidad multiplataforma **4**

7. Estructura del proyecto **6**

8. Capturas de la aplicación **7**

9. Compilación y ejecución **10**

9.1. Lanzadores por plataforma 10

9.1.1. macOS 10

9.1.2. Windows 10

9.1.3. Linux 11

9.2. Registros y diagnóstico 11

10. Resultados y Conclusiones **11**

11. Referencias **12**

1. Introducción

Tras haber consolidado la base de modelado geométrico en la etapa anterior, el proyecto avanza hacia la aplicación de propiedades ópticas y lumínicas para dotar de realismo a la escena del Día de Muertos. Cabe señalar que la mayoría de los modelos propios fueron remodelados para esta fase, demostrando un mayor compromiso con el proyecto y aumentando significativamente la calidad visual y geométrica de esta entrega.

En esta etapa, los modelos grises son transformados mediante la aplicación de mapas de texturas (Color, Rugosidad y Normales), la configuración de materiales transparentes, la creación de un entorno inmersivo (*Skybox*) y la implementación de un sistema de iluminación dinámica.

El desarrollo se mantuvo en C++17 utilizando OpenGL 3.3. Se programaron *shaders* específicos para procesar texturas PBR simplificadas e iluminar la escena mediante un modelo híbrido que combina la luz ambiental y direccional (Luna) con luces puntuales (cirios y faroles) y atenuación cuadrática. Todo el trabajo fue estructurado para mantener un rendimiento óptimo al procesar miles de instancias texturizadas simultáneamente.

2. Objetivos

2.1. Objetivo general

Implementar un sistema de texturizado, materiales e iluminación en tiempo real sobre la escena previamente modelada, logrando un acabado visual inmersivo que respete las proporciones geométricas y optimice los recursos en un entorno multiplataforma.

2.2. Objetivos específicos

- Diseñar y hornear texturas PBR (Color, Normales y Rugosidad) a partir de nodos procedurales en Blender.
- Implementar la carga dinámica de imágenes 2D con ImageIO en macOS y `stb_image` en Windows y Linux.
- Configurar un *Skybox* con texturas panorámicas para simular la transición lumínica del atardecer.
- Desarrollar *shaders* que calculen luces puntuales con atenuación y refracción óptica simulada para vidrios y líquidos.
- Garantizar la correcta compilación y ejecución en macOS, Windows y distribuciones Linux con soporte para OpenGL 3.3.

3. Alcance de la entrega

El alcance de esta fase comprende el texturizado de los modelos propios de la ofrenda y el entorno, la aplicación de materiales a los modelos de librería, la creación procedural de coordenadas UV, el mapeo de transparencias (recorte alfa) y la programación de los *shaders* de iluminación.

Se incluyen los mapas difusos, mapas de normales calculados en tiempo real mediante derivadas matemáticas de los fragmentos, canales de metalidad/rugosidad y el entorno atmosférico cúbico. También se incorpora la refracción óptica simulada a través de cálculos de transmitancia y *Fresnel* para jarrones y vasos.

Quedan fuera de esta entrega las animaciones esqueléticas complejas, las sombras dinámicas de alta resolución (*Shadow Mapping*) y las reflexiones especulares en tiempo real basadas en trazado de rayos.

4. Metodología de trabajo ágil

Para la administración de esta segunda etapa del proyecto, el equipo continuó iterando sobre el marco de trabajo *Scrum*, adaptando los *sprints* a las necesidades específicas de mapeo UV, programación de *shaders* y pruebas de rendimiento.

El *Product Backlog* se reestructuró enfocándose en el acabado visual y la optimización. El desarrollo se dividió en cuatro iteraciones principales (*sprints*):

1. **Definición y Diseño Procedural:** Asignación de propiedades de materiales, creación de redes nodales en Blender y horneado (*baking*) de texturas PBR.
2. **Programación Gráfica:** Desarrollo e integración de *shaders* para iluminación híbrida, cálculos de normales en tiempo real y transmisión óptica.
3. **Integración y Pruebas Multiplataforma:** Creación de rutinas nativas para carga de imágenes, validación del *Skybox* y resolución de conflictos de despliegue en macOS, Linux y Windows.
4. **Optimización y Refactorización:** Identificación de cuellos de botella lumínicos, retopología geométrica de modelos excedidos en polígonos y automatización mediante los lanzadores nativos.

Las reuniones de sincronización asíncronas permitieron identificar tempranamente los bloqueos arquitectónicos (como los conflictos del repositorio en Git por configuraciones específicas de SO), facilitando una toma de decisiones rápida por parte del *Scrum Master* y los desarrolladores para unificar el `CMakeLists.txt` y mantener la entrega en los plazos establecidos.

5. Diseño y desarrollo

5.1. Flujo de Trabajo en Blender: Texturas Procedurales

Para garantizar una alta calidad visual sin depender de fotografías externas, se optó por un flujo de trabajo procedural. Las texturas de los modelos propios fueron diseñadas directamente en el editor de nodos de Blender mediante funciones matemáticas de ruido y mezcla de colores.

Posteriormente, estas redes nodales complejas fueron horneadas (*baked*) a mapas de bits tradicionales (Color, Rugosidad y Normales). Finalmente, los modelos, junto con sus texturas, fueron exportados bajo el estándar glTF 2.0 y almacenados en la ruta `assets/models/propios/`. Este empaquetado aseguró una lectura eficiente al cargar la escena en OpenGL.

5.2. Mapeo de Normales y Materiales PBR

En lugar de cargar y precalcular los vectores tangentes en los vértices, el *fragment shader* (`escena.frag`) calcula las normales mapeadas en tiempo real. Esto se logró utilizando las funciones `dFdx` y `dFdy` sobre la posición de los vértices y las coordenadas UV, permitiendo extraer relieves altamente detallados para elementos como el pan de muerto, los muros de adobe y las calaveritas de azúcar de manera muy optimizada.

5.3. Generación Procedural de UVs y Recorte Alfa

Para el empedrado y la alfombra de cempasúchil, las coordenadas UV se generaron de forma procedural en el archivo `Camino.cpp` basándose en la longitud de arco de la spline Catmull-Rom. El *shader* procesa máscaras con recortes alfa (`uCorteAlfa`) para dar forma a los bordes dentados del papel picado y descartar los píxeles transparentes de los pétalos dispersos, reduciendo drásticamente la carga geométrica.

Para evitar la superposición o penetración geométrica de texturas complejas, se implementó una cuadrícula espacial de alturas en la clase `ApoyoDulce` (`ApoyoDulces.h`), la cual recorta cada triángulo contra la celda correspondiente garantizando un apilamiento perfecto y natural de los elementos.

5.4. Iluminación Híbrida y Transmisión

El modelo lumínico abandonó la luz direccional genérica de la fase de modelado por un sistema híbrido. Éste consiste en:

- **Luz direccional lunar:** Un tinte direccional tenue modificado con parámetros de oclusión ambiental.
- **Luces puntuales:** Un arreglo en el *shader* que soporta hasta 64 fuentes simultáneas (aplicado a cirios y veladoras) con atenuación cuadrática en base a un radio fijo.
- **Transmisión óptica:** Un cálculo aproximado sin refracción de pantalla utilizando el índice de refracción (`uIor`) y el cálculo de *Fresnel* para dotar de realismo a objetos de vidrio y agua, reforzando cantos y permitiendo visibilidad hacia atrás sin artefactos.

5.5. Entorno Atmosférico (Skybox)

Se integró un mapa cúbico para el cielo mediante la clase `Skybox`. El *vertex shader* del cielo elimina la traslación de la matriz de vista asegurando que el horizonte permanezca estático a medida que el jugador se mueve. En el *fragment shader* (`cielo.frag`), se extrae la luminancia del píxel (*luma*) para oscurecer dinámicamente el atardecer original y mezclarlo hacia una tonalidad nocturna inmersiva.

6. Retos arquitectónicos y compatibilidad multiplataforma

La validación se realizó en macOS, Windows y Linux. Además de los problemas visuales, se probaron la configuración con CMake, los compiladores, la localización de recursos y los lanzadores sin consola. Los incidentes relevantes y su solución final fueron los siguientes:

1. **Artefactos generados y divergencias en Git.** Los productos de CMake, Xcode y Visual Studio no son portables entre equipos. Se amplió `.gitignore` para excluir `build-*`, cachés, binarios, objetos, bibliotecas, registros y archivos propios del sistema operativo. La documentación y el código fuente permanecen versionados; las ramas divergentes se integraron mediante `git pull –rebase` para conservar tanto la documentación remota como las correcciones locales.

2. **Tamaño del *framebuffer* en pantallas de alta resolución.** En Retina y otras pantallas con escalado, las dimensiones lógicas de la ventana no coinciden con los píxeles renderizables. Se empleó `glfwGetFramebufferSize` antes de `glViewport`, evitando que la escena ocupara sólo una fracción de la ventana.

3. **Localización portable de recursos.** La aplicación no debe depender del directorio desde el que fue invocada. `Rutas.cpp` obtiene la ubicación real del ejecutable con `GetModuleFileNameW` en Windows y `/proc/self/exe` en Linux; después busca `assets/` y `shaders/` con `std::filesystem`. En Windows se usan rutas anchas para admitir caracteres UTF–8. Esto eliminó dependencias de separadores y directorios de trabajo específicos.

4. **Texturas visibles en macOS, pero ausentes en Windows y Linux.** La primera versión de `ImageLoader.cpp` sólo implementaba la decodificación bajo `#ifdef __APPLE__`; en las demás plataformas devolvía `false`. La geometría y los colores planos aparecían, pero fallaban PNG, JPEG y el *skybox*. La solución fue conservar ImageIO/CoreGraphics en macOS e integrar `stb_image` 2.30 para Windows y Linux. La cabecera se incluyó en `external/stb/` y se limitaron los decodificadores a PNG y JPEG. La despremultiplicación alfa de CoreGraphics se mantuvo para evitar bordes oscuros en macOS.

5. **Lanzador de macOS.** Usar directamente un script como `CFBundleExecutable` produjo fallos de apertura y resultados afectados por la caché de Finder. El paquete final utiliza un pequeño lanzador Mach–O nativo que localiza el proyecto y delega la compilación a `Contents/Resources/iniciar.zsh`. La aplicación permanece local y sin firma, por lo que la primera apertura puede requerir clic derecho, **Abrir**. Los detalles se guardan en `logs/lanzador-macos.log`.

6. **VBScript, PowerShell y Visual Studio en Windows.** La cadena que invocaba PowerShell desde VBScript quedó inicialmente sin terminar; se corrigió construyendo las comillas con `Chr(34)`. Después, el lanzador buscaba exclusivamente Visual Studio 2022 aunque el equipo de prueba tenía Visual Studio 2026. Ahora consulta `vswhere.exe`, selecciona automáticamente `Visual Studio 17 2022` o `Visual Studio 18 2026` y usa carpetas de compilación separadas.

7. **Diagnósticos incompletos y compilaciones simultáneas en Windows.** Con `ErrorActionPreference` configurado como `Stop`, Windows PowerShell interpretaba la primera línea de `stderr` de CMake como excepción y truncaba el diagnóstico. Se encapsularon las llamadas nativas para registrar toda la salida y evaluar `LASTEXITCODE`. Además, un doble clic durante una compilación oculta iniciaba otro MSBuild y causaba *Permission denied* sobre archivos `.obj`; un *mutex* nombrado impide ejecuciones concurrentes.

8. **Diferencias de compilador y coma flotante.** MSVC no aceptó el uso indirecto de `std::array`; se agregó explícitamente `¡array¿`. También se observaron 3055 instancias en Windows frente a una referencia de 3057 en macOS por diferencias de redondeo en pruebas geométricas con `float`. Una variación pequeña ahora se registra como aviso y una desviación mayor continúa deteniendo la carga, de modo que no se ocultan errores reales.

9. **OpenGL 3.3 dentro de una máquina virtual.** Windows 11 ARM bajo UTM compiló correctamente, pero su controlador VirtIO 3D expuso aceleración DirectX y no un contexto OpenGL 3.3 compatible. La aplicación falló al crear la ventana, mientras que el mismo repositorio funcionó en Windows físico con el controlador oficial de la GPU. Por tanto, la compilación dentro de una VM no garantiza el nivel de OpenGL requerido; debe verificarse la capacidad gráfica del entorno invitado.

7. Estructura del proyecto

El proyecto incorpora nuevos archivos y carpetas destinados a la administración de las texturas y los *shaders* actualizados. Notarás que la columna de Elementos se ha ampliado para mayor claridad.

Tabla 1: Archivos y carpetas nuevos o actualizados.

Elemento	Propósito
`src/render/Skybox.cpp`	Carga del mapa cúbico, manipulación del *luma* y renderizado esférico del cielo.
`src/render/ImageLoader.cpp`	Decodificación PNG/JPEG mediante ImageIO en macOS y `stb_image` en Windows/Linux.
`src/Rutas.cpp`	Localización del ejecutable y resolución portable de `assets/` y `shaders/`.
`src/render/Shader.cpp`	Analizador sintáctico para inyección de macros (`#define`) y compilación dinámica de GLSL.
`shaders/escena.frag`	Algoritmos de iluminación, derivadas parciales para relieves, transmitancia y variables PBR.
`shaders/escena.vert`	Transformación espacial adaptada para procesamiento de instancias texturizadas y tintes de color.
`assets/textures/`	Texturas base, mapas de color, metalidad/rugosidad generados procedimentalmente.
`assets/skybox/atardecer/`	Fragmentos de la imagen esférica para procesar el entorno panorámico de la escena.
`external/stb/stb_image.h`	Decodificador de imágenes de un solo encabezado incluido para Windows y Linux.
`ejecutables/`	Lanzadores y scripts silenciosos para macOS, Windows y Linux.

8. Capturas de la aplicación



Figura 1: Vista general de la ofrenda iluminada.

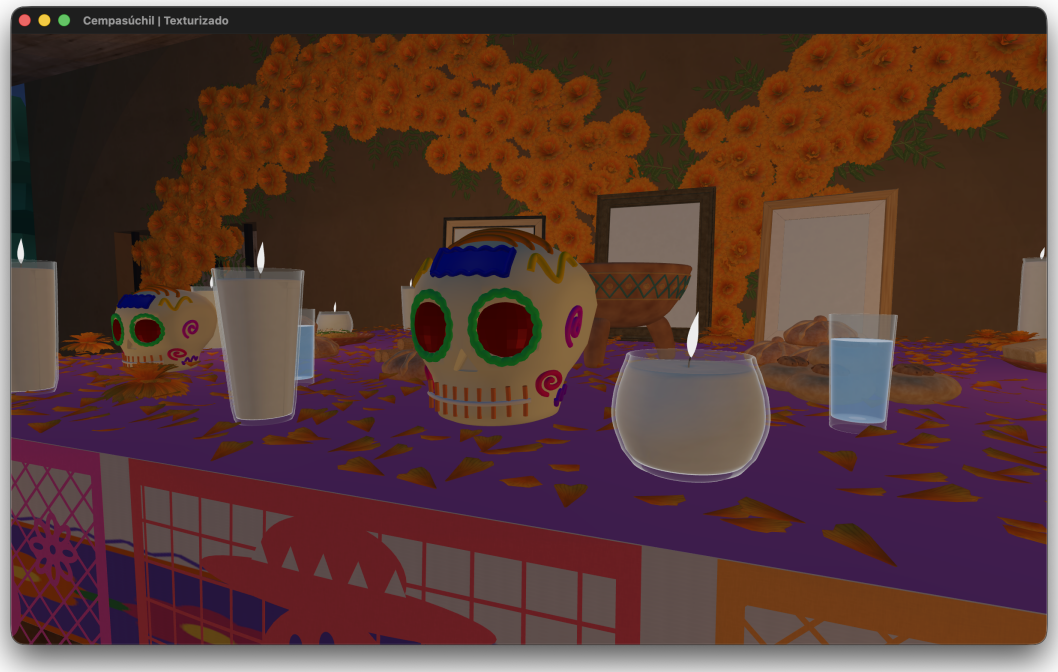


Figura 2: Detalle de las texturas PBR y efecto de luz puntual sobre los objetos.

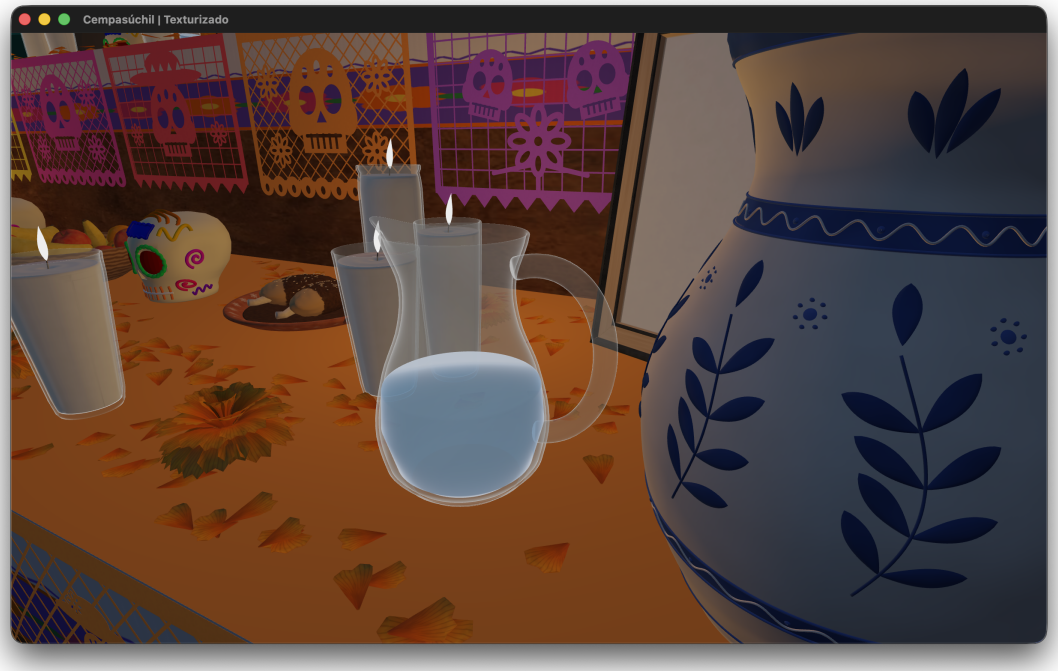


Figura 3: Aproximación óptica en materiales de vidrio y líquidos.

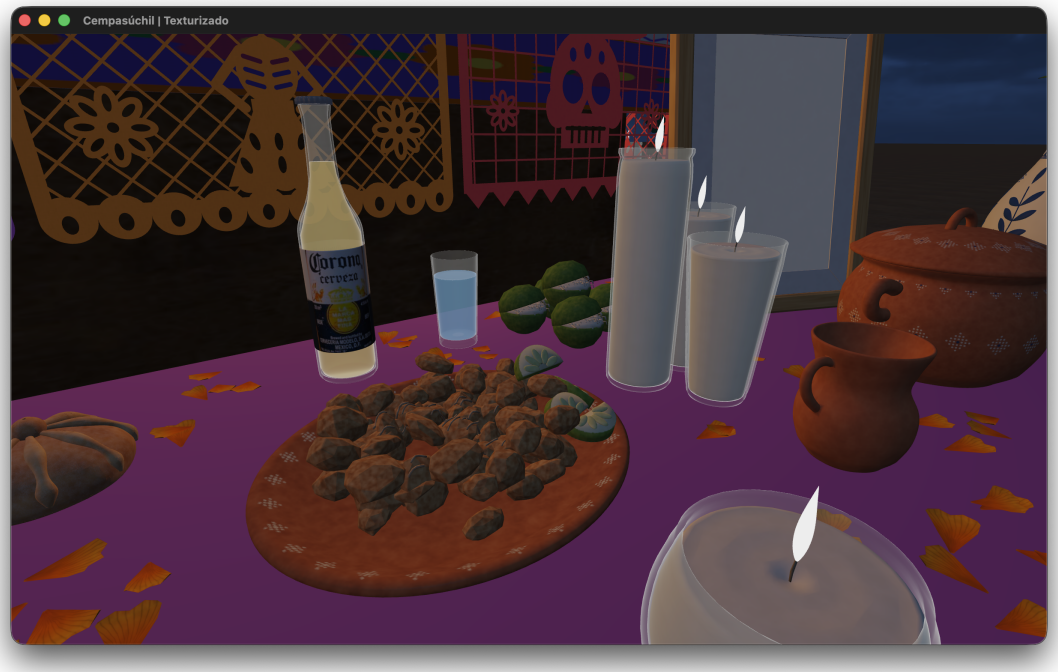


Figura 4: Aproximación óptica en materiales de vidrio y líquidos.

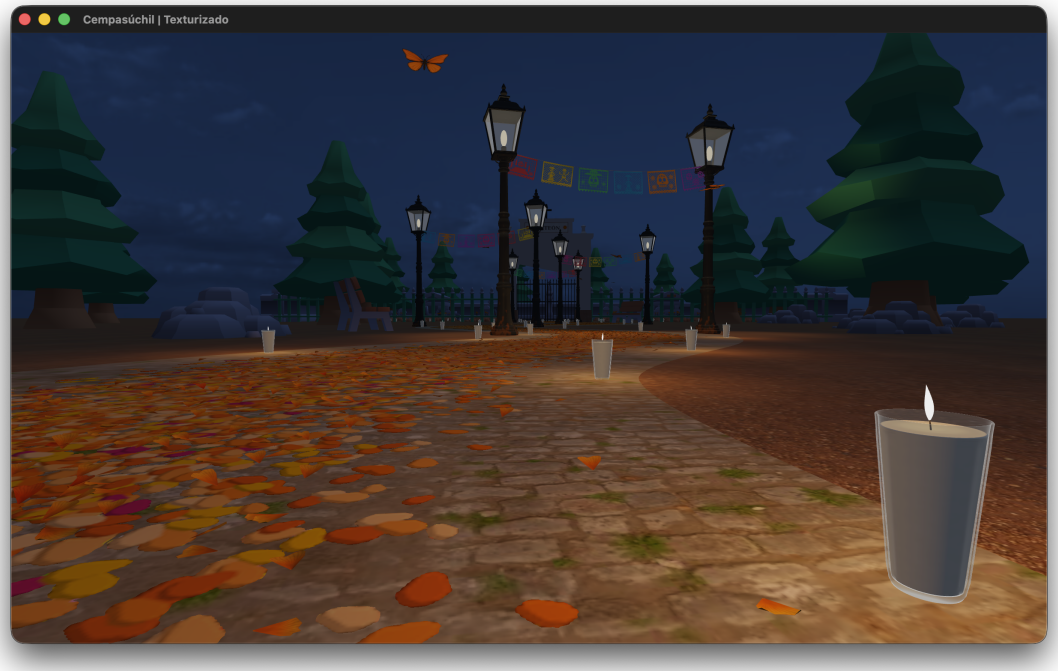


Figura 5: Camino empedrado procedural iluminado por faroles y veladoras.

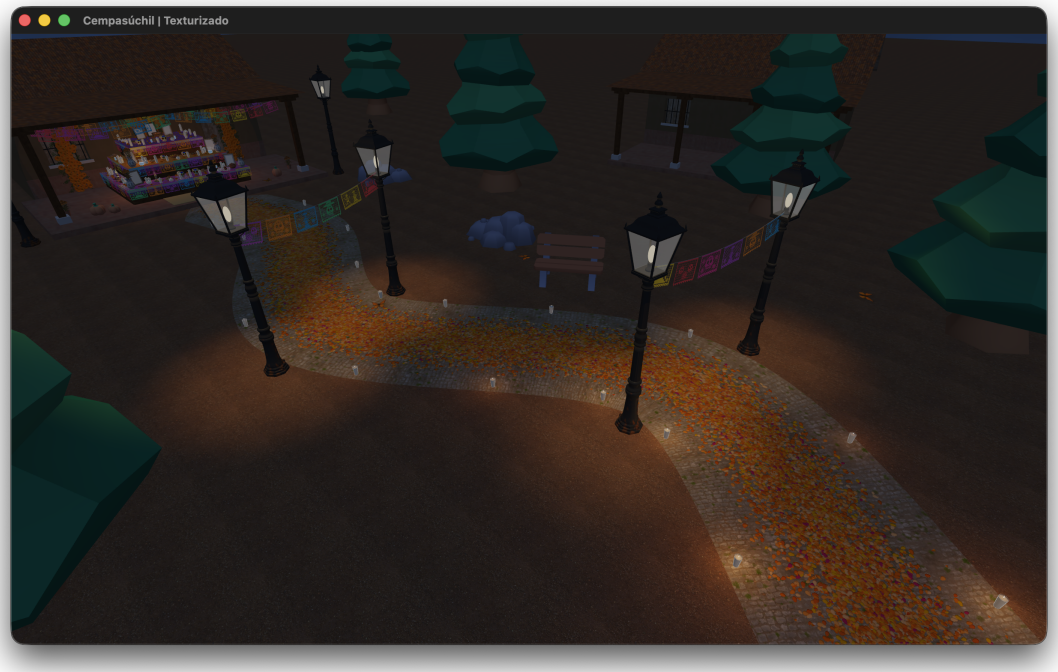


Figura 6: Vista general del camino iluminado.

9. Compilación y ejecución

La ejecución se validó en macOS, Windows físico y Linux. Todas las plataformas requieren una GPU o controlador que proporcione OpenGL 3.3; la presencia de aceleración 3D o DirectX por sí sola no satisface este requisito.

Para optimizar el flujo de trabajo y facilitar la revisión del proyecto, el equipo desarrolló lanzadores nativos para cada sistema operativo. Estos accesos directos automatizan por completo el ciclo de construcción y despliegue, estandarizando el proceso bajo las siguientes características:

- Configuran y compilan el proyecto a través de CMake estrictamente en modo `Release`.
- Actualizan y sincronizan los directorios de recursos (`assets` y `shaders`) hacia el directorio de salida.
- Ejecutan la escena automáticamente una vez finalizada la compilación sin procesos en terminal.
- Mantienen la consola oculta durante el proceso para ofrecer una experiencia gráfica limpia.
- Redirigen y escriben toda la salida técnica, advertencias y diagnósticos en la carpeta `logs/` ubicada en la raíz del proyecto.

9.1. Lanzadores por plataforma

El repositorio debe encontrarse en una carpeta local con permisos de escritura, pues cada lanzador crea su propio directorio `build-*`. Los lanzadores no descargan dependencias y se encuentran en `ejecutables/`.

9.1.1. macOS

Se requiere CMake, las herramientas de línea de comandos de Xcode y soporte del sistema para OpenGL 3.3. Tras clonar o descargar el repositorio, se abre:

```
ejecutables/macOS/TexturizadoCempasuchil-0.2.1.app
```

Por ser una aplicación académica local sin firma, Gatekeeper puede bloquear el primer intento; se debe usar clic derecho, **Abrir**. El lanzador busca CMake en PATH, Homebrew y la aplicación oficial de CMake, compila en `build-mac-app/` y conserva el último binario válido si una recompilación posterior falla.

9.1.2. Windows

Se requiere Windows de 64 bits, un controlador de GPU con OpenGL 3.3, CMake disponible en PATH y Visual Studio Community 2022 o 2026 con la carga **Desarrollo para el escritorio con C++**, MSVC x64/x86 y un Windows SDK. Después se abre:

```
ejecutables“windows“TexturizadoCempasuchil.vbs
```

VBScript inicia PowerShell sin consola; `vswhere` selecciona el generador instalado y compila para x64 en `build-windows-vs17/` o `build-windows-vs18/`. En Windows ARM64 el ejecutable x64 puede compilarse mediante las herramientas cruzadas, pero la ejecución sigue necesitando un controlador OpenGL compatible.

9.1.3. Linux

Se requiere CMake, un compilador C++17, OpenGL y los paquetes de desarrollo X11 utilizados por GLFW. En distribuciones basadas en Debian/Ubuntu se puede preparar el entorno con:

```
sudo apt update
sudo apt install build-essential cmake libgl1-mesa-dev “
   xorg-dev
```

En Arch Linux se emplean los paquetes equivalentes `base-devel`, `cmake`, `mesa` y `libx11`. Se marca el archivo `.desktop` como confiable/ejecutable y se abre, o se ejecuta:

```
chmod +x ejecutables/linux/iniciar˙linux.sh
./ejecutables/linux/iniciar˙linux.sh
```

9.2. Registros y diagnóstico

Los registros de cada plataforma son `logs/lanzador-macos.log`, `logs/lanzador-windows.log` y `logs/lanzador-linux.log`. Los mensajes internos de la escena se almacenan como `registro.txt` dentro del directorio de compilación. Si la aplicación informa que no puede crear la ventana, se debe comprobar la versión de OpenGL y actualizar el controlador del fabricante; reinstalar el compilador no corrige una limitación gráfica.

NOTA: Para mas detalles del funcionamiento interno de la automatización de estos lanzadores, revisar las dependencias exactas de cada sistema o consultar la ubicación de los archivos de registro y diagnóstico, se recomienda leer el archivo `README.md` ubicado en la ruta `ejecutables/README.md`.

10. Resultados y Conclusiones

La fase de texturizado e iluminación elevó significativamente el impacto visual y la inmersión del entorno del Día de Muertos. Sin embargo, la implementación del modelo de iluminación híbrida presentó un reto importante de rendimiento para el equipo. Al integrar las fuentes de luz dinámica, detectamos cuellos de botella severos debido a que algunos modelos exportados desde Blender poseían una densidad poligonal excesiva; el motor gráfico estaba calculando la iluminación por fragmento sobre una cantidad innecesaria de triángulos, saturando el procesamiento. La solución implementada fue optimizar estos modelos, reduciendo drásticamente su conteo de triángulos (retopología y diezmado) pero conservando una apariencia visual prácticamente idéntica a la original gracias al detalle aportado por los mapas de normales.

El uso de texturas PBR elaboradas procedimentalmente, acoplado al cálculo de normales derivado directamente en el *fragment shader*, demostró un excelente equilibrio entre calidad fotográfica y optimización algorítmica. Asimismo, el diseño arquitectónico multiplataforma fue crucial para superar con éxito barreras técnicas de librerías nativas, sistemas de archivos, repositorios e interfaces de alta resolución.

El correcto mapeo alfa para reducir polígonos (como en la alfombra de pétalos), la optimización geométrica de las mallas y el sistema de refracción simulada permiten afirmar que la entrega cumple satisfactoriamente con los parámetros visuales propuestos. Para la fase final, el entorno técnico

robusto y optimizado construido hasta el momento servirá como base ideal para integrar de forma fluida sistemas de partículas y animaciones.

11. Referencias

- Blender Foundation. (2026). *Blender 5.2*. https://www.blender.org/
- Khronos Group. (2021). *glTF 2.0 Specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html
- Khronos Group. (2010). *OpenGL 3.3 Core Profile Specification*. https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf
- Kitware. (2026). *CMake Documentation*. https://cmake.org/documentation/
- nothings/stb. (2025). *stb_image 2.30*. https://github.com/nothings/stb
- GLFW. (2026). *GLFW Documentation*. https://www.glfw.org/documentation.html